Advanced Computer Architecture

Unit I : Introduction to Parallel Processing

Dr. Roshan Koju

2024 December 8 / 15

Week I & II

Contents

Unit I: Introduction to Parallel Processing

1. Definition and significance of parallel processing.
2. Challenges in Parallel Processing
3 Flynn's Taxonomy
4 Feng's Classification of Parallel Computers
5 Architectural Classification and Applications of Parallel Processing
6 Cluster Computing.
7. Pipelining in Parallel Programming
8. Domain Decomposition vs. Functional Decomposition
9 Parallel Programming Models
10. Types of Parallel Algorithms
11 Performance of Parallel Algorithms
12 Parallel Programming Languages
13 Solving Problems with Parallel Algorithms

Definition of parallel processing

a computing technique in which multiple processors or computational units work simultaneously to execute multiple tasks or solve a single problem. In a parallel processing system:

1. The workload is divided into smaller units that can run simultaneously.
2. The subtasks communicate and synchronize with one another as required.
3. The goal is to achieve faster execution and improved resource utilization.



Parallel processing can be implemented using various architectures, including multicore processors, symmetric multiprocessing (SMP), massively parallel processors (MPP), or distributed computing systems.

Parallel processing is categorized by how data and tasks are divided and processed. These categories include:

1. Data Parallelism: Dividing data into smaller chunks and processing them simultaneously.
2. Task Parallelism: Assigning different tasks to separate processors for concurrent execution.
3. Hybrid Parallelism: Combining data and task parallelism to optimize performance

Significance of parallel processing

Performance Improvement:

A weather simulation task that might take hours on a single processor can be completed in minutes when distributed across multiple processors.

Scalability for Large Data Sets:

Search engines like Google use parallel processing to index and retrieve data across billions of web pages in real time.

Real-Time Processing:

In stock trading, algorithms process vast amounts of data to make buy/sell decisions in milliseconds.

Cost Efficiency with Resource Utilization:

Using distributed systems like cloud computing allows organizations to leverage parallel processing without investing heavily in dedicated hardware.

Enabling Advanced Technologies:

- **Machine Learning and AI:** Training neural networks requires processing millions of computations simultaneously.
- **Graphics Rendering:** High-quality 3D graphics and virtual reality rely on parallel GPU processing.
- **Scientific Research:** Simulations of physical phenomena like climate modeling and molecular dynamics depend on parallel systems.



Challenges of Parallel Processing

- **Task Division:** Identifying independent subtasks can be complex.
- **Synchronization and Communication:** Coordination between tasks can lead to overhead.
- **Scalability Issues:** Adding more processors doesn't always guarantee linear speedup due to communication delays and resource contention.



Flynn's classification

	Single data stream	Multiple data streams
Single instruction stream	**SISD** - sequential processing - possible concurrency via pipelining or superscalarity	**SIMD** - vectors/array processors - pipelined processors - associative processors - GPUs
Multiple instruction streams	**MISD** - Fault-tolerant systems - (potentially) systolic arrays	**MIMD** - shared memory model - distributed memory model - MPP systems, Clusters



Feng's classification

Mode	Computer Model	Degree of parallelism
WSPS N = 1 M = 1	The 'MINIMA'	(1,1)
WPBS N=1 M>1	STARAN MPP DAP	(1,256) (1,16384) (1,4096)
WSBP n>1 m=1 (Word Slice Processing)	IBM 370/168 UP CDC 6600 Burrough 7700 VAX 11/780	(64,1) (60,1) (48,1) (16/32,1)
WPBP n>1 m>1 (fully parallel Processing)	Illiav IV	(64,64)

In Feg's classification parallelism is exhibited (presented) by **word length(n)** and a **bit-slice length (m).**

- A bit slice is a string of bits one from each of the words at the same vertical position.
- World length is the number of bits in a word.



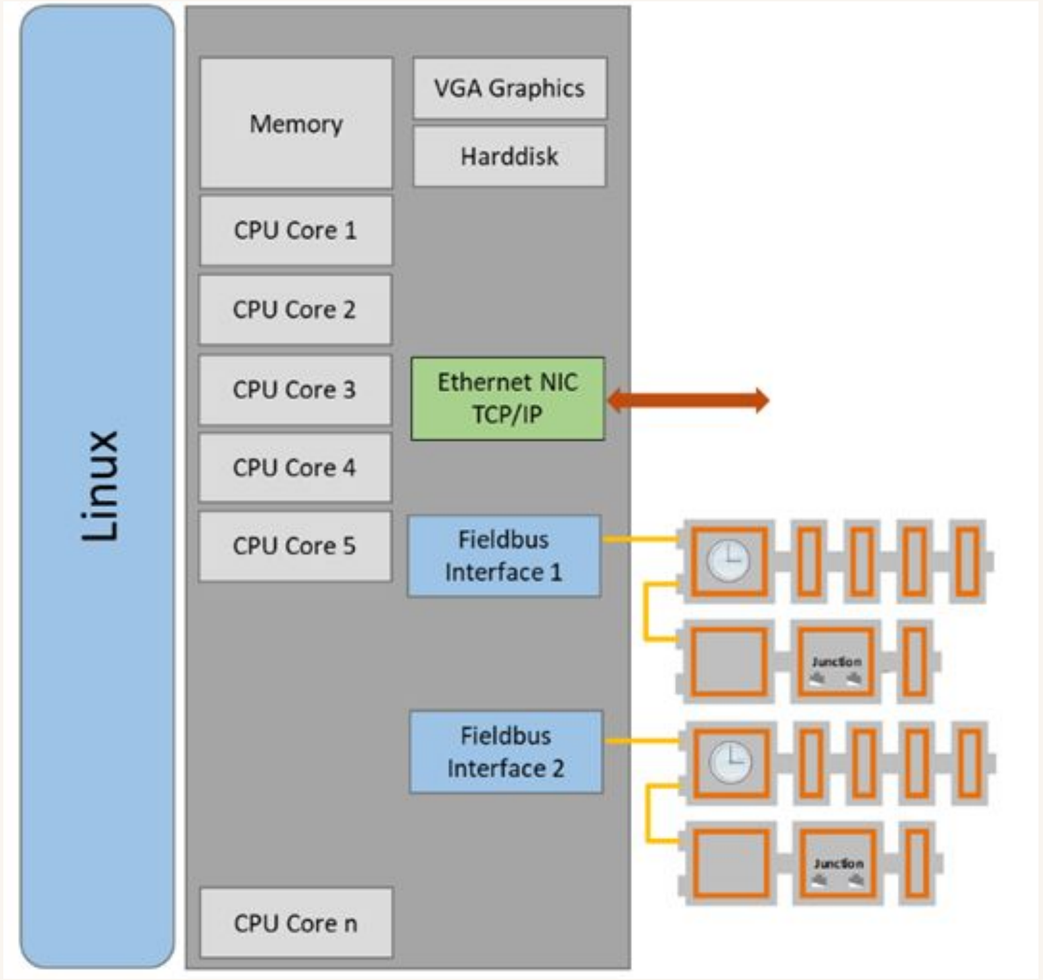
Architectural Classification of Parallel Processing

Symmetric Multiprocessing (SMP).

Symmetric Multiprocessing (SMP) is a type of computer architecture where multiple processors share a common memory and work collaboratively on tasks. SMP is widely used in systems that demand high performance, scalability, and reliability. Each processor in an SMP system runs its own instance of the operating system, but all processors share memory, I/O, and system resources equally.

- Symmetry Among Processors
- Shared Memory Architecture
- Shared Bus/System Interconnect
- Single Operating System
- Cache Usage
- Scalability
- Resource Sharing
- Fault Tolerance
- Performance Metrics
- Simplified Parallel Programming Model

8

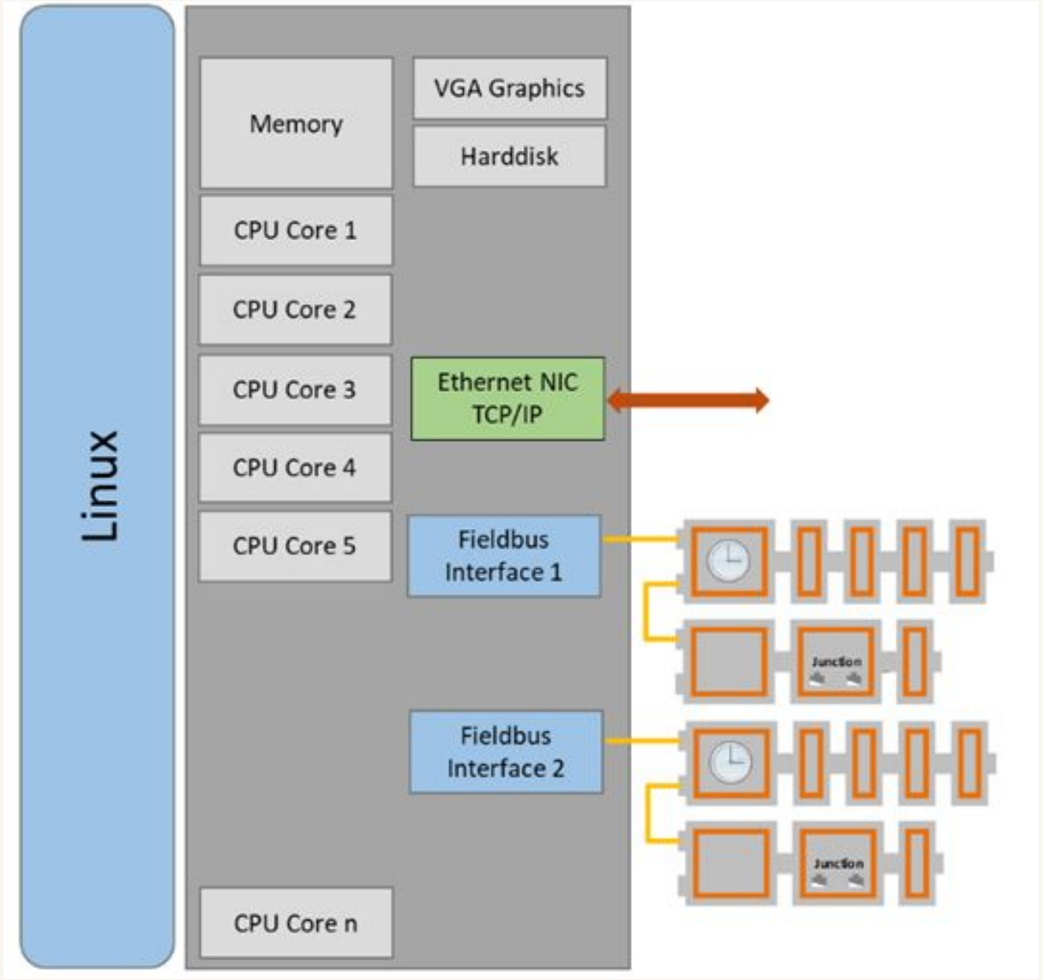


Architectural Classification of Parallel Processing

Symmetric Multiprocessing (SMP).

Symmetric Multiprocessing (SMP) is a type of computer architecture where multiple processors share a common memory and work collaboratively on tasks. SMP is widely used in systems that demand high performance, scalability, and reliability. Each processor in an SMP system runs its own instance of the operating system, but all processors share memory, I/O, and system resources equally.

- Symmetry Among Processors
- Shared Memory Architecture
- Shared Bus/System Interconnect
- Single Operating System
- Cache Usage
- Scalability
- Resource Sharing
- Fault Tolerance
- Performance Metrics
- Simplified Parallel Programming Model

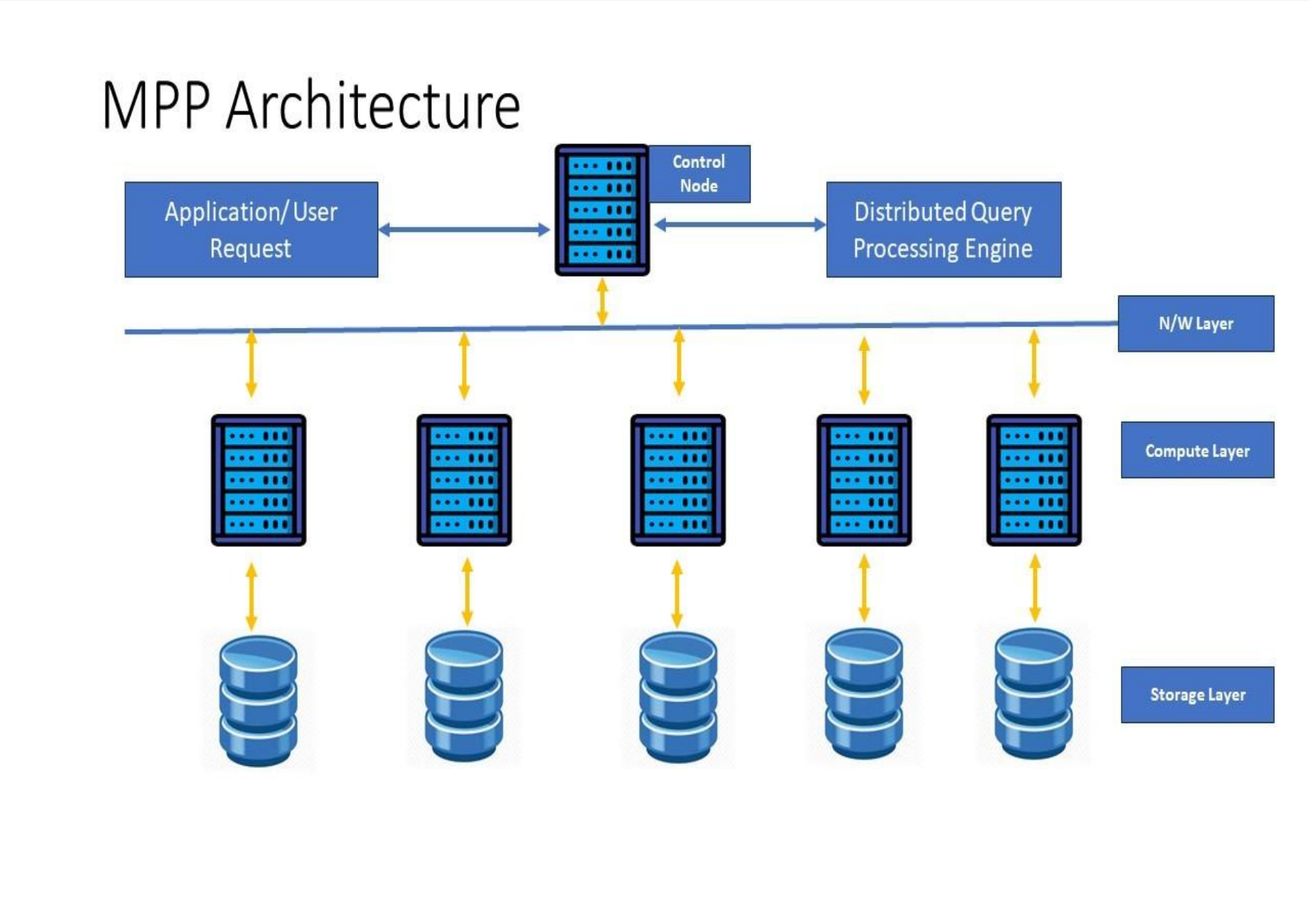


Architectural Classification of Parallel Processing

Massively Parallel Processing (MPP).

Massively Parallel Processing (MPP) refers to a computing architecture where numerous processors work simultaneously on different parts of a computational task. It is a scalable approach designed to handle large-scale, data-intensive problems, making it a cornerstone of high-performance computing (HPC).

a) MPP supports shared-nothing Architecture
b) In MPP each processor works on a different part of the task.
c) Each processor has its own set of disks
d) Each node is responsible for processing only the rows on its own disk
e) Scalability is easy by just adding nodes- Horizontal Scaling
f) Usually comes with huge compression ability
g) MPP processors communicate with each other a messaging interface
h) In MPP each processor uses its own operating system (OS) and memory.



Architectural Classification of Parallel Processing

Cluster Computing.

Cluster computing is a model of computing that involves connecting a group of independent computers, called nodes, to work together as a single system. These nodes collaborate to solve computational problems, providing high performance, scalability, and fault tolerance. It is widely used in fields requiring intensive computations, such as scientific research, data analysis, and financial modeling.

Architecture of Cluster Computing

Nodes

- Each node is an individual computer (e.g., PC, server) with its own CPU, memory, and storage.
- Nodes are typically uniform to ensure consistency in performance.

Interconnection Network

- Nodes are connected using a high-speed network, such as **Gigabit Ethernet**, **InfiniBand**, or **Fiber Channel**, to facilitate communication.
- The network's bandwidth and latency significantly impact the cluster's performance.

Cluster Middleware

- Software that manages the cluster, enabling coordination and resource allocation.

Storage System

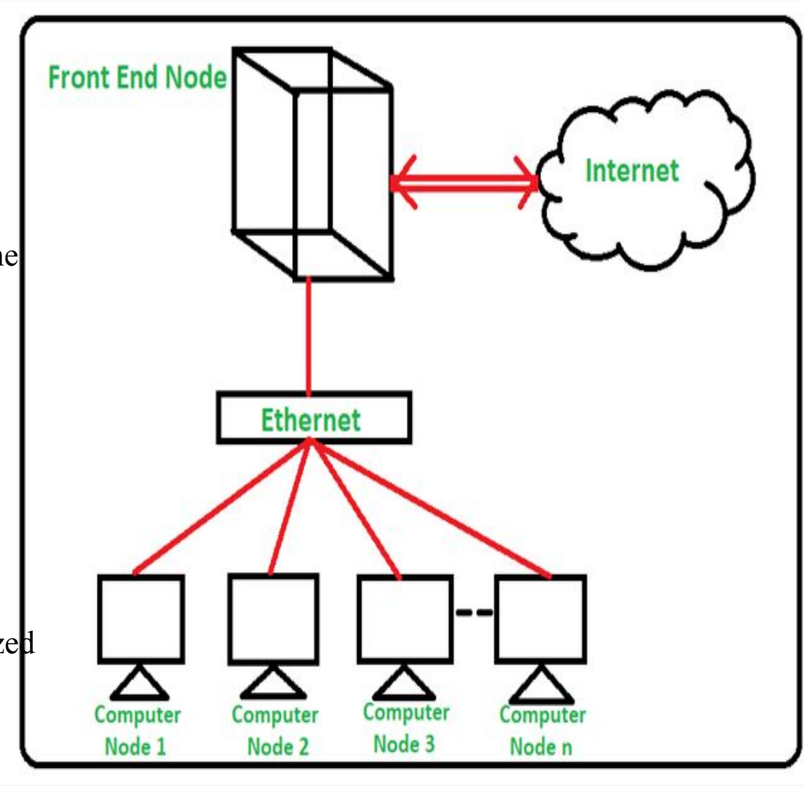
- A shared storage system is often used to allow nodes to access the same data.
- Common systems include **Network File System (NFS)** and distributed file systems like **HDFS (Hadoop Distributed File System)**.

Master Node

- Responsible for scheduling tasks, managing resources, and monitoring the cluster's health.
- Centralized systems have a single master node, while decentralized systems distribute responsibilities across multiple nodes.

Compute Nodes

- Perform the actual computational tasks assigned by the master node.



Architectural Classification of Parallel Processing

Cluster Computing.

Cluster computing is a model of computing that involves connecting a group of independent computers, called nodes, to work together as a single system. These nodes collaborate to solve computational problems, providing high performance, scalability, and fault tolerance. It is widely used in fields requiring intensive computations, such as scientific research, data analysis, and financial modeling.

Classification of Cluster :

1. Open Cluster :

IPs are needed by every node and those are accessed only through the internet or web. This type of cluster causes enhanced security concerns.

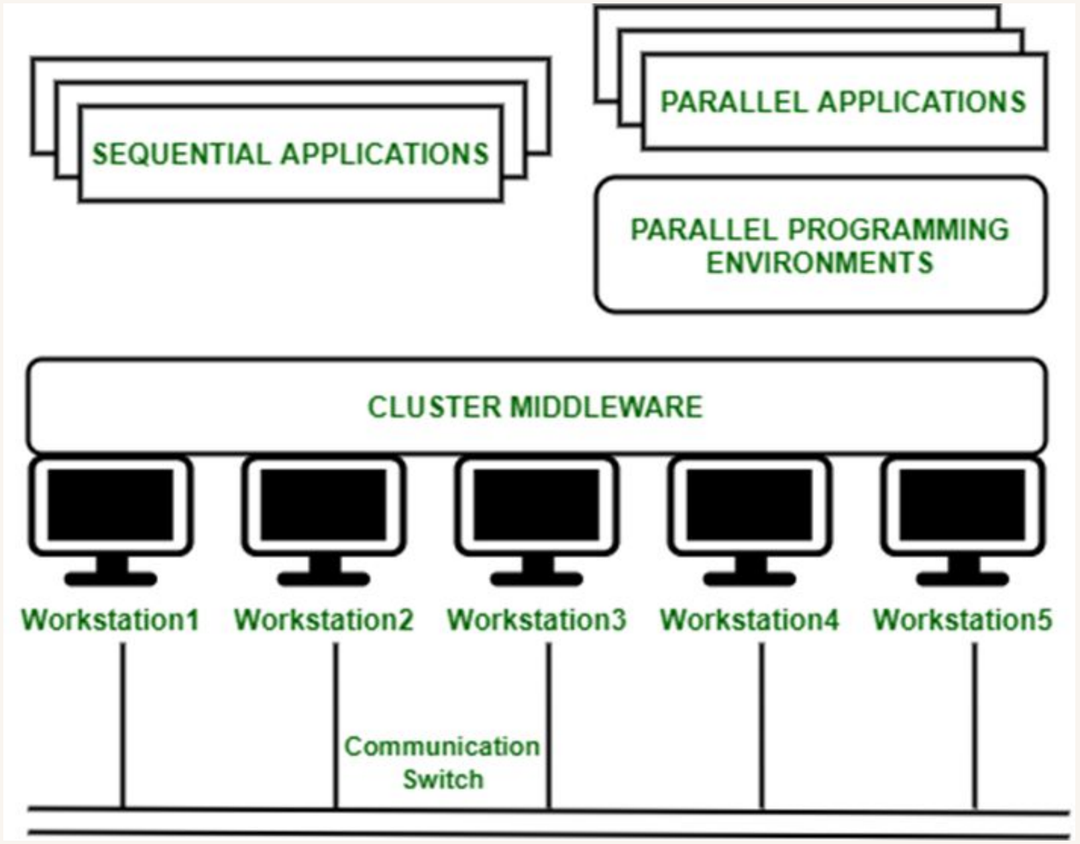
2. Close Cluster :

The nodes are hidden behind the gateway node, and they provide increased protection. They need fewer IP addresses and are good for computational tasks.

1. Load-balancing clusters: Workload is distributed across multiple installed servers in the cluster network.

2. High availability (HA) clusters: A collection group that maintains very high Availability. Computers pulled from these systems are considered to be very much reliable and may not face downtime, even possibly in any instance.

3. High-performance (HP) clusters: This computer networking tactic use supercomputers and Cluster computing to resolve complex and highly advanced computation problems



Characteristics of Parallel Algorithms

1. **Decomposition and Concurrency:**
   - Parallel algorithms decompose a problem into smaller tasks that can execute simultaneously. This decomposition can be data-based (data parallelism) or task-based (task parallelism). For example, splitting a matrix multiplication problem into independent sub-matrix operations.
2. **Communication:**
   - Parallel algorithms often require processors to exchange intermediate data during execution. The communication overhead, depending on the architecture (shared or distributed memory), influences the algorithm's efficiency.
3. **Synchronization:**
   - Coordination among processes is crucial to ensure correct execution. Synchronization mechanisms (e.g., locks, barriers) manage dependencies but can introduce delays.
4. **Scalability:**
   - A good parallel algorithm scales efficiently with the number of processors, maintaining or improving performance as more resources are added.

13

5. **Load Balancing:**
   - Effective distribution of tasks among processors ensures that no processor remains idle while others are overloaded. Poor load balancing leads to suboptimal performance.
6. **Fault Tolerance:**
   - Parallel algorithms should handle processor failures gracefully, particularly in distributed systems, ensuring continued execution without significant data loss.
7. **Efficiency and Speedup:**
   - Efficiency is the ratio of speedup to the number of processors used. Speedup measures how much faster a parallel algorithm is compared to its sequential counterpart. A well-designed parallel algorithm maximizes both.
8. **Overhead:**
   - Parallel algorithms incur overhead due to communication, synchronization, and data distribution. Minimizing this overhead is crucial for achieving high performance.

Parallel Programming Techniques

Task Decomposition Techniques

Task decomposition (also known as functional decomposition) divides a problem into distinct tasks, each representing a functional operation that can be executed concurrently

Key Features:

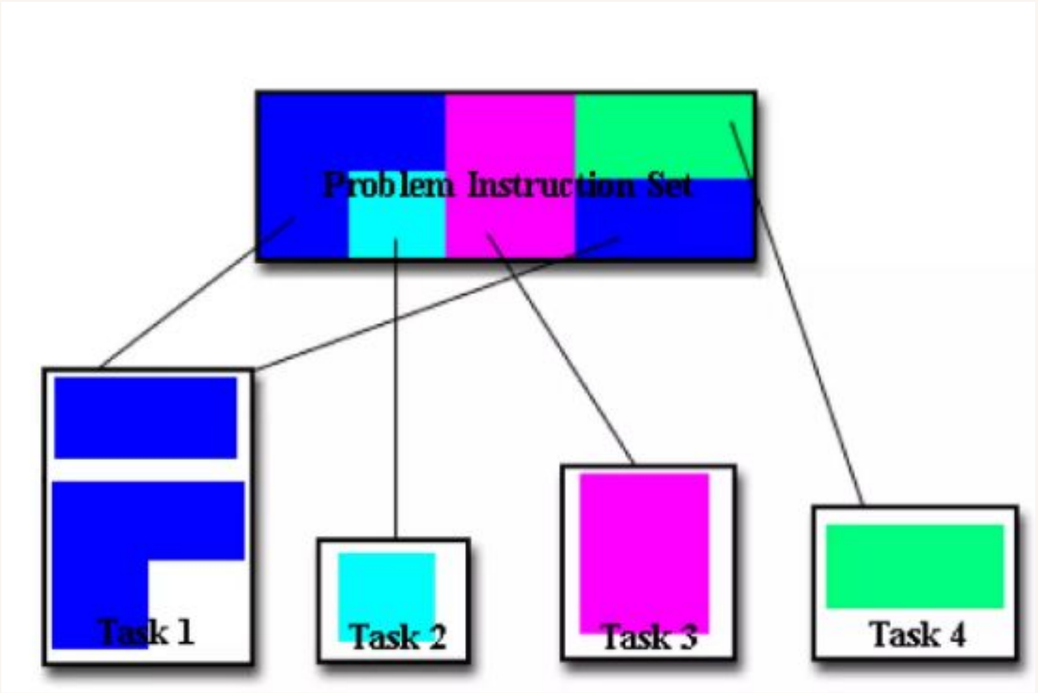
- Tasks may have varying computational workloads.
- Task dependencies determine execution order.
- Typically used when tasks involve distinct processes (e.g., data fetching, processing, visualization).

Steps in Task Decomposition:

1. **Identify Tasks**: Break the application into logically distinct operations.
2. **Analyze Dependencies**: Determine dependencies among tasks to avoid conflicts.
3. **Assign Tasks to Processes**: Allocate tasks to processors or threads based on their dependencies and workload.

Example:

- In a weather simulation application:
  - Task 1: Fetch weather data from sensors.
  - Task 2: Process the fetched data using a mathematical model.
  - Task 3: Visualize the results for end users.
    Each task can run in parallel, provided proper synchronization.



Parallel Programming Techniques

Data Decomposition Techniques

Data decomposition (or data partitioning) divides the data into smaller chunks that can be processed simultaneously by different processing units.

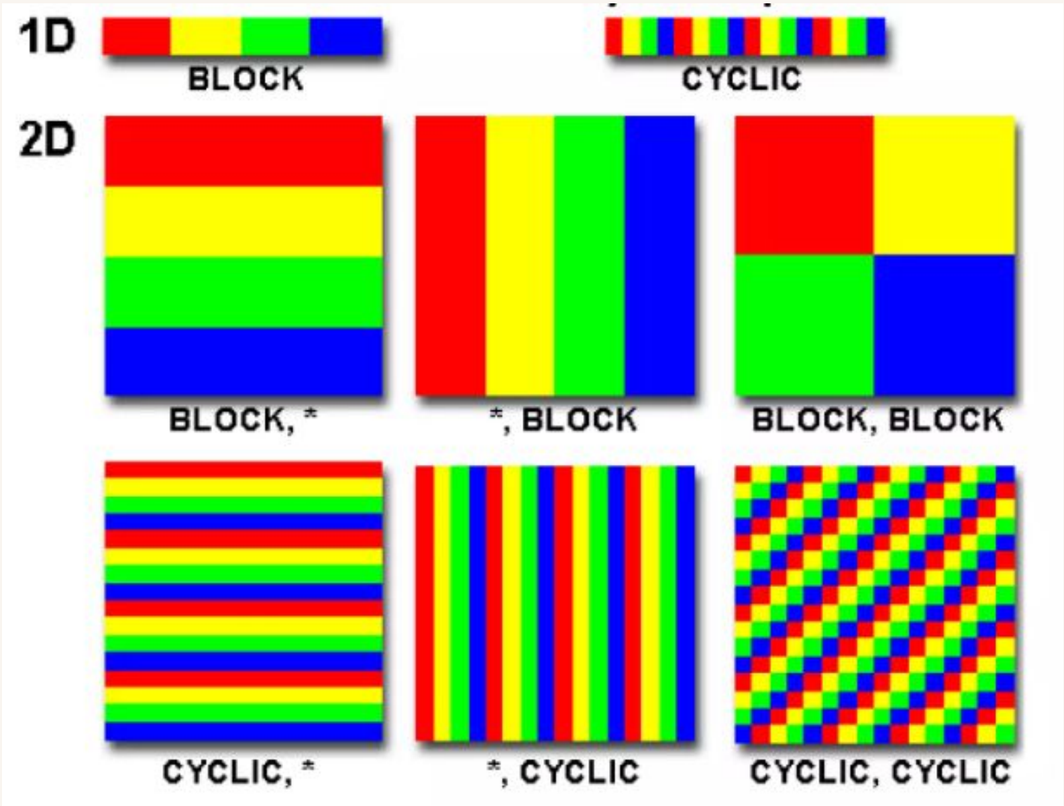
Key Features:

- Best suited for problems where the same operations are performed on different subsets of data.
- Improves scalability and efficiency in data-intensive tasks.

Types of Data Decomposition:

1. **Block Decomposition**: Data is divided into contiguous blocks and assigned to different processors.
   - Example: Dividing a 1D array into equal-sized chunks for processing.
2. **Cyclic Decomposition**: Data elements are distributed cyclically among processors.
   - Example: Assigning every nth element of an array to the same processor.
3. **Block-Cyclic Decomposition**: A hybrid of block and cyclic decomposition.
   - Example: Dividing data into small blocks and distributing cyclically.

15



Example:

In matrix multiplication:

- Divide the rows of matrix A and columns of matrix B into smaller blocks.
- Assign each block to different processors for simultaneous computation of the result matrix.

Pipelining in Parallel Programming

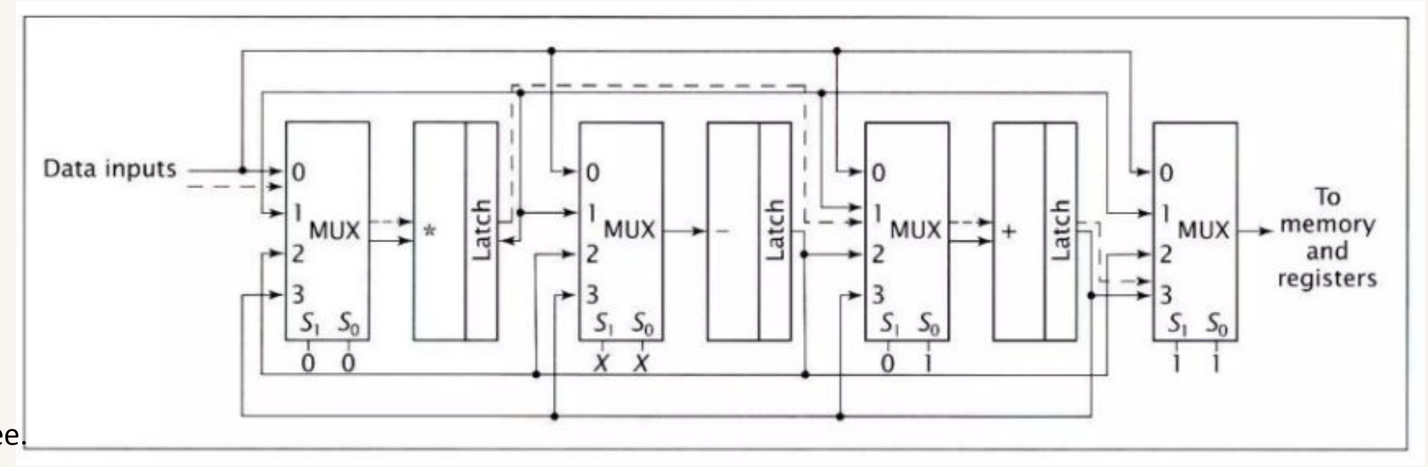
Pipelining divides a task into a sequence of subtasks, where the output of one becomes the input for the next. Each subtask is processed in parallel, creating an assembly line effect.

Key Characteristics:

- Suitable for problems where tasks are divided into stages with dependencies.
- Improves throughput by overlapping execution of subtasks.
- Reduces idle time of processing units.

Stages in Pipelining:

1. **Divide Work into Stages**: Identify sequential subtasks.
2. **Parallelize Stages**: Assign each stage to a processor.
3. **Execute Concurrently**: Start new data inputs as soon as the previous stage is free.



Pipeline Performance Metrics:

1. **Throughput**: Number of tasks completed per unit time.
2. **Latency**: Time taken to process a single task from start to finish.

16

Parallel Programming Models

Parallel programming models provide a framework for designing and implementing software that can execute tasks concurrently on multiple processing units. These models abstract away some of the complexities of hardware interaction, enabling developers to focus on application-level parallelism.

Shared Memory Model (OpenMP)

The shared memory model is a parallel programming paradigm where multiple threads execute within the same memory space. This model is particularly suited for systems with shared memory architecture, such as symmetric multiprocessors (SMP).

Key Features:

- **Global Address Space:** All threads share the same memory, which eliminates the need for explicit data transfer.
- **Thread-Based Execution:** Computations are divided among threads, which can access shared variables.
- **Synchronization Mechanisms:** Required to prevent race conditions and ensure consistent data (e.g., locks, barriers).

Core Components of OpenMP:

1. **Directives:** Pragma-based syntax for marking parallel regions (e.g., `#pragma omp parallel`).
2. **Runtime Library Routines:** Functions for managing threads, synchronization, and performance.
3. **Environment Variables:** Control runtime behavior (e.g., `OMP_NUM_THREADS` for setting the number of threads).

Advantages:

- Simplicity in coding due to implicit communication.
- Suitable for fine-grained parallelism.

17

Message Passing Interface (MPI):

MPI is the most popular standard for distributed memory parallel programming. It provides a set of libraries for communication and synchronization across multiple processes.

Core Components of MPI:

1.Point-to-Point Communication: Direct communication between two processes (e.g., MPI_Send and MPI_Recv).
2.Collective Communication: Communication involving all processes in a group (e.g., broadcast, scatter, gather).
3.Communicators: Define groups of processes that can communicate.

Advantages:

- High scalability; suitable for large-scale systems like clusters and supercomputers.
- Flexibility to run on heterogeneous systems.

Disadvantages:

- Increased complexity due to explicit communication.
- Overhead from data transfer and synchronization.

Parallel Algorithms for Multiprocessors

Parallel algorithms leverage multiple processors to perform computations simultaneously, enabling faster execution and efficient resource utilization. Multiprocessor systems, characterized by shared or distributed memory architectures, are ideal platforms for implementing such algorithms.

Divide-and-Conquer Algorithms

Divide-and-conquer is a strategy that breaks a problem into smaller subproblems, solves these subproblems concurrently, and combines their results to form the solution to the original problem.

Steps in Divide-and-Conquer:

1. **Divide:** Partition the input data into disjoint subsets.
2. **Conquer:** Solve each subset independently, often in parallel.
3. **Combine:** Merge the results of the subproblems.

Applications in Parallel Processing:

- **Sorting:** Parallel quicksort, parallel merge sort.
- **Matrix Operations:** Strassen's matrix multiplication.
- **Numerical Problems:** Parallel prefix sum, FFT (Fast Fourier Transform).

Example: Parallel Merge Sort

1. Divide the array into two halves.
2. Sort each half concurrently using recursive calls.
3. Merge the two sorted halves using parallel merging techniques.

Graph-Based Algorithms

Graph-based algorithms use the structure of a graph to parallelize computations. These algorithms are crucial in solving problems like shortest path, network flow, and graph traversal.

Key Graph-Based Parallel Algorithms:

1. **Parallel Breadth-First Search (BFS):** Used for traversing graphs level by level. Each level can be explored concurrently by multiple processors.
2. **Minimum Spanning Tree (MST):** Algorithms like Borůvka's MST can be parallelized by processing independent components simultaneously.
3. **Parallel Dijkstra's Algorithm:** Suitable for shortest-path computation in distributed memory systems.

Graph Representation:

- **Adjacency Matrix:** Easy to parallelize but memory-intensive.
- **Adjacency List:** Efficient for sparse graphs but requires sophisticated load-balancing strategies.

18

Performance of Parallel Algorithms

Speedup

Speedup is a measure of how much faster a parallel algorithm runs compared to its sequential counterpart. It is defined as the ratio of the time taken to execute the sequential algorithm to the time taken to execute the parallel algorithm.

$$\text{Speedup (S)} = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}$$

Where:

- $T_{\text{sequential}}$ is the execution time of the algorithm in sequential execution.
- T_{parallel} is the execution time of the algorithm using multiple processors.

Efficiency

Efficiency measures how effectively the processors are utilized in the parallel algorithm

$$\text{Efficiency (E)} = \frac{\text{Speedup}}{p} = \frac{T_{\text{sequential}}/T_{\text{parallel}}}{p}$$

Where:

- p is the number of processors used.

19

Scalability

Scalability refers to the ability of a parallel algorithm to effectively use more processors as they are added. There are two types of scalability:

- **Strong scalability**: How the execution time decreases as the number of processors increases, for a fixed problem size.
- **Weak scalability**: How the algorithm handles increasing problem sizes as the number of processors increases.

A parallel algorithm is said to be scalable if increasing the number of processors leads to a proportional decrease in execution time or a proportional increase in problem size without significant performance degradation.

Parallel Programming Languages

MPI (Message Passing Interface)

MPI is a standardized and portable message-passing system designed for parallel programming in distributed-memory systems, where each processor has its own local memory. It is one of the most widely used models for high-performance computing (HPC) applications.

- **Key Features:**
  - MPI allows communication between processes running on different nodes or machines.
  - It provides routines for sending and receiving messages between processes, which can be on the same machine or distributed across different machines.
  - MPI supports both **point-to-point communication** (i.e., between two processes) and **collective communication** (i.e., involving multiple processes).
  - It is language-independent but typically used with languages like C, C++, and Fortran.

MPI is designed to be highly scalable, making it suitable for supercomputers, clusters, and grid computing

OpenMP (Open Multi-Processing)

OpenMP is a parallel programming model for shared-memory systems, where multiple processors or cores share the same memory space. OpenMP simplifies the process of parallel programming by allowing developers to parallelize existing sequential programs with minimal changes.

- **Key Features:**
  - OpenMP uses **compiler directives**, **runtime library routines**, and **environment variables** to control parallelism.
  - It supports **multi-threading**, where each thread operates on a shared memory space.
  - It is most commonly used in C, C++, and Fortran, but the focus is on adding parallelism to sequential code with simple annotations.
  - OpenMP allows for **implicit parallelism** where loops or sections of code can be automatically parallelized using a few simple commands.

CUDA (Compute Unified Device Architecture) for GPUs

CUDA is a parallel computing platform and programming model developed by NVIDIA for general-purpose computing on GPUs (Graphics Processing Units). CUDA is designed to accelerate computations by harnessing the massive parallelism of modern GPUs.

- **Key Features:**
  - CUDA allows developers to write programs that execute parallel code on NVIDIA GPUs.
  - It supports a programming model based on **threads** and **blocks**, where threads are grouped into blocks, and blocks are organized into grids.
  - CUDA provides APIs for working with large datasets, matrix computations, and other highly parallel tasks.
  - It supports both **CPU-GPU parallelism** and **GPU-only parallelism** for data-intensive tasks.

Solving Problems with Parallel Algorithms

Load Balancing Issues
Load balancing refers to the process of distributing tasks or computations evenly across available processors in a parallel system. The goal is to ensure that no processor is underutilized or overwhelmed, which would lead to inefficiencies and wasted resources.
Challenges:
Uneven Task Distribution: If the work is not evenly distributed, some processors may finish their tasks early while others may be overwhelmed. This leads to idle processors and slower overall performance.
Dynamic Workloads: In some cases, the amount of work assigned to each processor may change during execution, especially when dealing with problems that involve dynamic inputs or evolving data (e.g., real-time processing).
Strategies for Load Balancing:
Static Load Balancing: Tasks are divided evenly at the start of the computation and remain fixed throughout. This works well when the size of each task is known ahead of time and does not change during execution.
Dynamic Load Balancing: Work is distributed dynamically during execution. This is more adaptable, especially when tasks take varying amounts of time to complete. A processor that finishes its task early can pick up additional work from other processors that are still busy.
Work Stealing: A technique where idle processors "steal" tasks from busy processors to balance the load dynamically.

Deadlock and Synchronization Challenges
Deadlock occurs in parallel systems when two or more processes are blocked forever, waiting for each other to release resources. In a parallel environment, deadlock is a serious issue because it halts the progress of tasks and results in wasted computational power.
Causes of Deadlock:
Resource Allocation: When multiple processes request resources (e.g., memory, I/O), and each process holds one resource while waiting for another.
Circular Waiting: A situation where processes form a cycle of dependencies, causing each process to wait for another in the cycle.
Prevention of Deadlock:
Resource Allocation Strategies: One approach is to allocate resources only if all required resources are available, ensuring no partial allocations occur.
Avoidance Algorithms: These algorithms ensure that a system does not enter a deadlock state by preventing circular wait conditions.
Timeouts: Implementing timeouts for waiting processes can help detect and recover from potential deadlocks.
Synchronization Challenges:
Synchronization refers to the coordination of tasks and the correct order of operations in a parallel system. Without proper synchronization, parallel tasks may access shared resources at the wrong time, leading to inconsistent results or race conditions.

Race Condition:

A race condition occurs when two or more processes access shared data simultaneously, and the final result depends on the order of execution. This can lead to unpredictable behavior, where the outcome may vary each time the program is run.

Synchronization Strategies:

Locks and Mutexes: These are mechanisms that allow only one process to access a shared resource at a time.

Semaphores: A signaling mechanism that controls access to shared resources, using signals to indicate whether resources are available or not.

Barriers: A synchronization method where processes wait until all of them reach a certain point before proceeding.

Practice Questions-1

1. Define parallel processing and discuss its significance in modern computing environments.
2. Discuss the advantages, limitations, and scenarios where parallel processing outperforms sequential approaches with examples
3. Compare and contrast Flynn's taxonomy with Feng's classification of parallel computers. How does Feng's framework enhance the understanding of parallel processing systems?
4. Critically evaluate the role of parallel processing in emerging technologies such as quantum computing and distributed systems.
5. Compare MPP and cluster computing.
6. Explain and compare multitasking, multiprocessing and multithreading.

7. Parallel processing is widely used in various domains. Identify three real-world applications of parallel processing and explain how its characteristics make it suitable for these applications.
8. What are the primary challenges associated with implementing parallel processing systems? Discuss trade-offs between complexity, cost, and performance in parallel computer architecture design.
9. Describe the evolution of classifications for parallel processing systems from traditional models like Flynn's taxonomy to hybrid models used in contemporary systems (e.g., cloud computing and GPUs).
10. Analyze the significance of parallel processing in improving system performance. Discuss the relationship between the degree of parallelism and scalability in parallel computing architectures.

Practice Questions-2

1. Explain the architecture of Symmetric Multiprocessing (SMP) systems. Discuss the advantages and challenges of using SMP for shared-memory applications. How does SMP ensure memory consistency in concurrent processes?
2. Compare and contrast Symmetric Multiprocessing (SMP) and Massively Parallel Processing (MPP) architectures. In which scenarios would MPP systems outperform SMP systems, and why? Provide examples of applications suited for MPP systems.
3. Define cluster computing and its role in modern computing environments. Discuss the design considerations for a high-performance cluster, including node configuration, network topology, and fault tolerance mechanisms.
4. Analyze the differences between fog computing and cloud computing in terms of architecture, latency, scalability, and resource management. Provide real-world examples where fog computing is more advantageous than cloud computing and justify your reasoning.
5. Describe the key characteristics of grid computing and how it differs from cluster and cloud computing. Discuss how resource allocation and job scheduling are managed in grid computing environments, using specific algorithms or techniques as examples.

6. Explain the architecture of Symmetric Multiprocessing (SMP) and discuss how it handles memory contention in multi-core processors. Provide an example of an application where SMP systems are preferable and justify your choice.

7. Explain how parallel processing is implemented in Hyperconverged Infrastructure (HCI) and traditional Three-Tier Computing. Discuss the following aspects:

   a. Resource pooling and scalability.
   b. Data locality and its impact on parallel processing performance.
   c. Challenges in maintaining fault tolerance and load balancing in both architectures.

   Provide a detailed comparison and suggest scenarios where each architecture would be most suitable for high-performance parallel computing workloads.

8. With the rise of hybrid cloud deployments, how can parallel processing models adapt to leverage the benefits of both HCI and three-tier architectures? Propose a hybrid framework combining strengths of both systems for optimal parallel processing.

10. Compare Open Clustering vs closed clustering.

11. Explain various synchronization methods used in SMP and MPP.

Practice Questions-3

1. Explain the difference between data decomposition and task decomposition in parallel programming. Provide examples where each technique would be most suitable.
2. Compare and contrast the shared memory model with the distributed memory model in parallel programming. Highlight the challenges associated with each.
3. Explain the working of a parallel merge sort algorithm for multiprocessor systems. Provide a pseudocode representation.
4. Discuss the key design considerations for implementing matrix multiplication on a multiprocessor system. Include aspects such as load balancing and communication.
5. What is Amdahl's Law, and how does it impact the scalability of parallel algorithms? Illustrate your answer with an example.Define speedup and efficiency in the context of parallel algorithms. How do they help in evaluating the performance of a parallel algorithm?
6. Discuss the features of OpenMP that make it suitable for parallel programming. How does it differ from MPI in terms of application scenarios?

7. Explain how CUDA facilitates parallel programming for GPUs. Provide an example of a simple CUDA kernel for vector addition.
8. A dataset contains 1 billion integers, and you need to find the maximum value using a parallel algorithm. Design an approach using a divide-and-conquer method and discuss its complexity.
9. Consider a scenario where a large dataset needs to be processed for image recognition. Analyze and compare the efficiency of task decomposition and data decomposition techniques for this problem. Highlight the impact of inter-process communication overhead, synchronization issues, and load balancing on the overall performance.
10. A parallel matrix multiplication algorithm is implemented on a multiprocessor system using distributed memory architecture. Evaluate the performance of this algorithm using metrics such as speedup, efficiency, and scalability. Additionally, discuss how the performance would be affected if the number of processors is doubled but the problem size remains constant (strong scaling).

Advanced Computer Architecture

Unit I

The End

Dr. Roshan Koju

2024 December 8/15

Week I & II